

SolarPredict

Solar Energy Forecasting & Microgrid Optimization System
Final Year Project — Complete Defense Guide
Douala & Maroua, Cameroon | XGBoost Model II | FastAPI + React + SQLite

What this document covers:

- Project purpose, motivation, and scope
- Complete folder structure — every file explained
- Backend deep-dive: FastAPI routes, services, database, ML pipeline
- Frontend deep-dive: React pages, components, API client
- Machine learning: data, features, model training, comparison
- All API endpoints with examples
- SQLite database schema and migration logic
- Microgrid optimization algorithm step by step
- Alert system (FR-7) and report generation (FR-9)
- Tests you should be able to demonstrate and run
- 60+ expected defense questions with model answers

Table of Contents

- Chapter 1** — Project Overview & Motivation
- Chapter 2** — Complete Folder Structure
- Chapter 3** — Backend — Core Files Explained
- Chapter 4** — Backend — Services Layer
- Chapter 5** — Machine Learning Pipeline
- Chapter 6** — Frontend — Pages & Components
- Chapter 7** — API Endpoints Reference
- Chapter 8** — Database Schema & Queries
- Chapter 9** — Microgrid Optimization Algorithm
- Chapter 10** — Alert System (FR-7)
- Chapter 11** — Report Generation (FR-9)
- Chapter 12** — How to Run the Project
- Chapter 13** — Tests to Run & Demonstrate
- Chapter 14** — Defense Questions & Model Answers

Chapter 1 — Project Overview & Motivation

Section 1.1 — What is SolarPredict?

SolarPredict is a full-stack web application built as a Final Year Project. It forecasts hourly solar irradiance (Global Horizontal Irradiance — GHI, measured in W/m^2) for two Cameroonian cities — **Douala** and **Maroua** — using a trained **XGBoost machine-learning model (Model II)**. The system also includes a rule-based **microgrid optimization engine** that simulates hourly battery charging, discharging, and grid import/export decisions based on the predicted solar output.

The application is split into a **FastAPI Python backend** and a **React + Vite frontend**. The backend exposes a REST API; the frontend renders live dashboards, charts, history tables, and downloadable reports.

Section 1.2 — Why This Project?

Cameroon has substantial solar energy potential, particularly in the Far North region (Maroua). However, solar energy is intermittent — it depends on weather conditions. Without accurate forecasting, operators of small solar microgrids cannot decide in advance when to charge batteries, when to rely on the grid, or when to export surplus energy. SolarPredict addresses this problem by providing a real-time prediction platform that:

- Fetches live weather data from the Open-Meteo free API
- Engineers 21 domain-specific features from raw weather and datetime inputs
- Runs an XGBoost model trained on NASA POWER historical data (2000–2022)
- Translates irradiance predictions into actionable microgrid operating instructions
- Stores predictions in SQLite for historical analysis
- Generates seasonal comparison reports and downloadable weekly PDFs

Section 1.3 — Two Cities, Two Climates

Attribute	Douala	Maroua
Region	Littoral (Coastal)	Far North
Latitude / Longitude	4.05°N, 9.77°E	10.60°N, 14.32°E
Climate Type	Equatorial — hot & humid	Sahelian — hot & dry
Dry Season	December – February	October – May
Rainy Season	March – November	June – September
Solar Potential	Moderate (coastal clouds)	High (clear skies, $>5 \text{ kWh/m}^2/\text{day}$)
Model File	douala_xgb_model_ii.pkl	maroua_xgb_model_ii.pkl

Section 1.4 — Functional Requirements

FR	Requirement	Where Implemented
FR-1	Fetch real-time weather data	backend/app/weather.py
FR-2	Engineer Model II features from weather	backend/app/features.py
FR-3	Run XGBoost prediction for any hour	backend/app/model_loader.py
FR-4	Save predictions to SQLite	backend/app/database.py
FR-5	Show hourly forecast on the frontend	frontend/src/pages/Prediction.jsx
FR-6	Simulate microgrid battery/grid schedule	backend/app/microgrid.py
FR-7	Alert when solar is low and battery is low	backend/app/services/alert_service.py
FR-8	Live dashboard with GHI chart and SoC gauge	frontend/src/pages/Dashboard.jsx
FR-9	Seasonal comparison report + CSV/PDF export	backend/app/services/report_service.py
FR-10	Select city (Douala or Maroua)	All pages — city dropdown

Chapter 2 — Complete Folder Structure

The project root is **SolarPredict/**. Every folder and file has a specific role. Understanding this structure is essential for the defense.

```
SolarPredict/
├── backend/                                ← Python FastAPI server
│   ├── app/
│   │   ├── __init__.py                    ← Makes app/ a Python package
│   │   ├── main.py                        ← FastAPI app, all REST routes
│   │   ├── config.py                      ← Cities, paths, feature list, thresholds
│   │   ├── database.py                    ← SQLite CRUD (init, save, query)
│   │   ├── features.py                    ← Model II feature engineering (21 features)
│   │   ├── weather.py                     ← Open-Meteo API client with 5 min cache
│   │   ├── model_loader.py                ← Load/cache XGBoost .pkl, run inference
│   │   ├── microgrid.py                   ← Battery/grid hourly simulation (FR-6)
│   │   ├── schemas.py                     ← Pydantic request/response models
│   │   └── services/
│   │       ├── __init__.py
│   │       ├── alert_service.py           ← FR-7 alert evaluation
│   │       ├── analytics_service.py       ← Model metrics + seasonal stats
│   │       ├── microgrid_recommendation.py ← Rule engine (High/Moderate/Low)
│   │       ├── report_service.py          ← Seasonal CSV report (FR-9)
│   │       └── weekly_report_service.py   ← 7-day PDF report generation
│   ├── requirements.txt                    ← Python dependencies
│   └── solar_predict.db                    ← SQLite database (auto-created)
├── frontend/                              ← React + Vite web application
│   ├── src/
│   │   ├── main.jsx                       ← React root entry, BrowserRouter
│   │   ├── App.jsx                        ← Route definitions
│   │   └── api/
│   │       ├── client.js                  ← fetch() wrappers for all API endpoints
│   │       └── components/
│   │           ├── Layout.jsx             ← Sidebar + shell
│   │           ├── StatCard.jsx           ← Reusable metric tile
│   │           ├── RecommendationBadge.jsx ← Colored microgrid advice badge
│   │           ├── AlertBanner.jsx        ← FR-7 alert banner
│   │           └── BatteryGauge.jsx        ← Visual SoC progress bar
│   └── pages/
│       ├── Dashboard.jsx                  ← FR-8 live dashboard
│       ├── Prediction.jsx                 ← Manual prediction runner
│       ├── History.jsx                     ← SQLite history table
│       ├── Microgrid.jsx                   ← FR-6 optimization UI
│       ├── Reports.jsx                     ← FR-9 seasonal + weekly reports
│       └── About.jsx                       ← Static project description
│   └── utils/
│       └── microgridSettings.js            ← localStorage sync for microgrid params
│   └── styles/
│       └── index.css                       ← Global dark-theme CSS
├── index.html                             ← Vite HTML entry point
└── package.json                           ← npm dependencies
```

- ■■■ vite.config.js ← Vite build config + /api proxy
-
- Model/ ← Trained XGBoost model files
- ■■■ douala_xgb_model_ii.pkl ← Douala XGBoost Model II
- ■■■ maroua_xgb_model_ii.pkl ← Maroua XGBoost Model II
- ■■■ douala_rf_model_ii.pkl ← Random Forest (for comparison)
- ■■■ maroua_rf_model_ii.pkl
- ■■■ douala_lstm_model_ii.pkl ← LSTM (for comparison)
- ■■■ maroua_lstm_model_ii.pkl
-
- Data/
- ■■■ Raw/
- ■ ■■■ Douala_Dataset.csv ← Raw NASA POWER data (2000–2022)
- ■ ■■■ Maroua_Dataset.csv
- ■■■ Processed/
- ■■■ Douala_Cleaned.csv
- ■■■ Maroua_Cleaned.csv
- ■■■ Douala_Features_Model_II.csv ← 21-feature training dataset
- ■■■ Maroua_Features_Model_II.csv
-
- training/ ← Jupyter notebooks for ML training
- ■■■ Model_training_Douala_Model_II.ipynb
- ■■■ Model_training_Maroua_Model_II.ipynb
- ■■■ Model_comparism_Model_II.ipynb
- ■■■ results_douala_model_ii.csv ← MAE/RMSE/R² results
- ■■■ results_maroua_model_ii.csv
- ■■■ model_ii_comparison_summary.csv ← Best model per city
-
- Data_Preprocessing.ipynb ← Raw → Cleaned data pipeline
- Exploratory_analysis.ipynb ← EDA: distributions, correlations
- Feature_engineering.ipynb ← Feature creation logic (Model I)
- feature_model_II.ipynb ← Feature creation for Model II
- Model_validation.ipynb ← Final model validation
- pyproject.toml ← Python project metadata
- vercel.json ← Vercel deployment config
- README.md ← Quick start guide

Chapter 3 — Backend Core Files Explained

3.1 — config.py

Purpose: Single source of truth for all configuration. No magic numbers are scattered across the codebase — everything lives here.

Variable / Constant	What It Stores
BASE_DIR	Absolute path to project root, computed with pathlib.Path so it works on any machine
MODEL_DIR	Points to SolarPredict/Model/ where the .pkl model files live
DB_PATH	Full path to solar_predict.db inside backend/
CITIES dict	Two keys: 'douala' and 'maroua'. Each has name, latitude, longitude, timezone, and model_file
MODEL_II_FEATURES	Ordered list of 21 feature names the XGBoost model was trained on. ORDER MATTERS.
TARGET	ALLSKY_SFC_SW_DWN — the NASA POWER variable being predicted (W/m ²)
BATTERY_SOC_MIN_PCT	0.20 (20%) — battery never discharges below this (FR-6)
BATTERY_SOC_MAX_PCT	0.95 (95%) — battery never charges above this (FR-6)
BATTERY_SOC_DEFAULT_PCT	0.50 (50%) — starting state of charge for simulations
ALERT_FORECAST_HOURS	6 — look 6 hours ahead to compute cumulative solar (FR-7)
ALERT_SOLAR_THRESHOLD_KWH_M2	0.5 — if 6 h solar < 0.5 kWh/m ² , low solar alert triggers (FR-7)
ALERT_BATTERY_SOC_PCT	40.0 — alert also requires battery SoC < 40% (FR-7)
DEFAULT_BATTERY_CAPACITY_KWH	20.0 — default for dashboard simulation
DEFAULT_DAILY_LOAD_KWH	35.0 — default daily energy demand
DEFAULT_PANEL_AREA_M2	12.0 — default solar panel area in square metres
DEFAULT_PANEL EFFICIENCY	0.18 — 18% panel efficiency (typical commercial panel)

3.2 — database.py

Purpose: All SQLite operations. Handles creating the table, migrating from old schemas, inserting records, and querying them. Uses Python's built-in sqlite3 module.

Function	What It Does
init_db()	Called at FastAPI startup. Creates the predictions table if it doesn't exist, or migrates it if columns differ from the expected schema.

<code>_ensure_predictions_schema(conn)</code>	Internal helper. Compares existing columns with expected columns and runs ALTER TABLE migration if needed.
<code>get_connection()</code>	Context manager that opens a sqlite3 connection with Row factory so rows can be accessed like dicts.
<code>save_predictions(city, records)</code>	Inserts a batch of forecast hours (list of dicts) into the predictions table using executemany for efficiency.
<code>get_prediction_history(city, limit)</code>	Selects up to 'limit' rows ordered by newest first. Optional city filter. Used by History page.
<code>get_predictions_last_days(city, days)</code>	Returns predictions from the last N days ordered oldest first. Used by weekly report service.
<code>get_latest_prediction(city)</code>	Returns the most recent single row. Used by dashboard to avoid duplicate saves.
<code>get_dashboard_stats(city)</code>	Aggregates: total count, count per city, 5 latest rows, average prediction per city.

Schema Migration Logic:

The database was designed with forward migration in mind. If the table already exists but has different columns (e.g., from an earlier version that used 't2m' instead of 'temperature'), the code renames the table to `predictions_legacy`, creates the new schema, copies data with column renaming, and drops the legacy table. This means you can upgrade without losing old data.

3.3 — weather.py

Purpose: HTTP client for the Open-Meteo free weather API. Fetches live hourly forecast data and transforms it into model-ready feature rows.

Component	Explanation
<code>OPEN_METEO_URL</code>	https://api.open-meteo.com/v1/forecast — the only external API used
<code>WEATHER_CACHE_TTL_SEC</code>	300 seconds (5 minutes). Weather data is cached in memory to avoid hammering the API on every dashboard refresh.
<code>_weather_cache</code> dict	Keys are (city, forecast_hours) tuples. Values are (timestamp, records) pairs.
<code>_http_client</code>	Single shared <code>httpx.AsyncClient</code> instance. Async because FastAPI uses <code>asyncio</code> .
<code>fetch_hourly_weather(city, hours)</code>	Main function. Returns a list of dicts — one per forecast hour. Each dict has: <code>datetime</code> , <code>reference_ghi</code> , <code>weather</code> (display fields), <code>features</code> (all 21 model inputs).
Open-Meteo variables fetched	<code>temperature_2m</code> (°C), <code>relative_humidity_2m</code> (%), <code>wind_speed_10m</code> (m/s), <code>precipitation</code> (mm), <code>shortwave_radiation</code> (W/m ² — used as reference GHI on chart)

wind_speed_unit='ms'	Forces Open-Meteo to return wind speed in m/s to match NASA POWER WS10M units used in training.
Reference GHI	Open-Meteo's shortwave_radiation is stored as reference_ghi and shown as a comparison line on the Prediction page chart (the 'ground truth' vs model).

3.4 — features.py

Purpose: Feature engineering. Converts raw weather + datetime into the 21 features the XGBoost model expects. This is the most important preprocessing step.

The 21 Model II Features:

Feature	Type	Description
YEAR	Raw	Calendar year from datetime
MO	Raw	Month (1–12)
DY	Raw	Day of month (1–31)
HR	Raw	Hour of day (0–23)
T2M	Raw	Temperature at 2m height (°C) from Open-Meteo
RH2M	Raw	Relative Humidity at 2m (%) from Open-Meteo
PRECTOTCORR	Raw	Precipitation (mm) — renamed from Open-Meteo precipitation
WS10M	Raw	Wind speed at 10m (m/s) from Open-Meteo
hour_sin	Cyclic	$\sin(2\pi \times \text{HR} / 24)$ — captures circular nature of hours
hour_cos	Cyclic	$\cos(2\pi \times \text{HR} / 24)$ — 23:00 is close to 00:00 in this encoding
month_sin	Cyclic	$\sin(2\pi \times \text{MO} / 12)$ — captures circular nature of months
month_cos	Cyclic	$\cos(2\pi \times \text{MO} / 12)$
day_sin	Cyclic	$\sin(2\pi \times \text{day_of_year} / 365)$
day_cos	Cyclic	$\cos(2\pi \times \text{day_of_year} / 365)$
day_of_year	Derived	Day number from 1 (Jan 1) to 365
is_daytime	Binary	1 if $6 \leq \text{HR} \leq 18$, else 0. Solar only active during day.
is_rainy	Binary	1 if PRECTOTCORR > 0, else 0
rain_log1p	Transform	$\log(1 + \text{PRECTOTCORR})$. Handles skewed rain distribution.
ws10m_log1p	Transform	$\log(1 + \text{WS10M})$. Handles skewed wind distribution.
season_dry	Binary	1 if dry season for the city (Douala: Dec–Feb; Maroua: Oct–May)
season_rainy	Binary	1 if rainy season. Opposite of season_dry.

Why cyclic encodings?

Linear encoding of hour (0–23) has a problem: the model sees hour 23 and hour 0 as far apart (distance = 23), but they are actually adjacent (both are late night/early morning). Using sin/cos transforms maps hours onto a unit circle, so 23 and 0 are close in feature space. This is standard practice in time-series ML.

Why log1p transforms?

Rainfall and wind speed are heavily right-skewed — most values are small (0–2 mm) but occasionally very large (50+ mm). $\log(1 + x)$ compresses large values without losing small ones ($\log(1+0) = 0$, so zero rain stays zero). This helps tree models split more efficiently.

3.5 — model_loader.py

Purpose: Loads the serialized XGBoost model from disk and caches it in memory. Runs inference (prediction) on a feature DataFrame.

Component	Explanation
_models dict	In-memory cache. Keys are 'douala' / 'maroua'. After first load, the model stays in RAM. No disk I/O for subsequent requests.
get_model(city)	Checks cache first. If not loaded, finds the .pkl file path from config.CITIES, loads with joblib.load(), stores in cache, returns it.
predict(city, feature_df)	Calls model.predict(df) and clamps all negative outputs to 0.0. Solar irradiance cannot be negative, so this is a physical constraint, not a bug fix.
joblib.load()	joblib is the standard way to serialize/deserialize scikit-learn and XGBoost models. It is faster and more memory-efficient than pickle for large numeric objects.

3.6 — schemas.py

Purpose: All Pydantic models. These define the shape of API request bodies and response payloads. FastAPI uses them for automatic validation, documentation, and serialization.

Schema Class	Used By	Purpose
PredictionRequest	POST /api/predict body	city (str) + hours (int 1–168)
WeatherSnapshot	PredictionPoint	7 weather fields for one hour
MicrogridAdvice	PredictionPoint, dashboard	status, recommendation, level
PredictionPoint	PredictionResponse	One forecast hour: datetime, prediction, weather, microgrid
PredictionResponse	POST /api/predict	Full response: city, model name, hours, list of PredictionPoint
DashboardResponse	GET /api/dashboard/basic	Lightweight version for simple dashboard

MicrogridRequest	POST /api/microgrid/optimize	city, battery params, hours (1–168)
MicrogridHour	MicrogridResponse.schedule	One hour of microgrid simulation
MicrogridResponse	POST /api/microgrid/optimize	Full simulation: summary + hourly schedule
ForecastPoint	EnhancedDashboardResponse	Simplified (datetime, prediction) for the 24h chart
AlertResponse	EnhancedDashboardResponse	FR-7 alert: active, message, severity, priorities
ModelMetrics	EnhancedDashboardResponse	Operational stats from stored predictions
SeasonBucket	SeasonalComparison	avg_prediction_wm2 + sample_count for one season
SeasonalComparison	Dashboard + Reports	Dry vs rainy season comparison
EnhancedDashboardResponse	GET /api/dashboard	Full FR-8 payload with all sub-objects
SeasonalReportResponse	GET /api/reports/seasonal	FR-9 seasonal report with export hint

Pydantic's role: When FastAPI receives a POST /api/predict request, it automatically deserializes the JSON body into a PredictionRequest object and raises a 422 Unprocessable Entity error if any field is missing or the wrong type. When the route returns an EnhancedDashboardResponse object, Pydantic automatically converts it to JSON. This removes all manual validation/serialization code.

Chapter 4 — Backend Services Layer

The **services/** folder contains business logic separated from the main routing file. This is good software engineering practice (Separation of Concerns).

4.1 — `microgrid_recommendation.py`

A pure function that takes a predicted irradiance value (W/m^2) and returns a **MicrogridRecommendation** dataclass. No database access, no I/O — just logic.

Condition	Status	Recommendation
$\text{prediction} > 700 \text{ W/m}^2$	High Solar Energy	Charge batteries and prioritize solar generation
$400 \leq \text{prediction} \leq 700 \text{ W/m}^2$	Moderate Solar Energy	Hybrid operation mode
$\text{prediction} < 400 \text{ W/m}^2$	Low Solar Energy	Use battery reserves or grid support

The **level** field ('high', 'moderate', 'low') is used by the frontend RecommendationBadge component to apply green/orange/red color styling.

4.2 — `alert_service.py` (FR-7)

Implements the two-condition alert system. The alert is **critical** (active) only when *both* conditions are true simultaneously:

- Cumulative 6-hour solar forecast $< 0.5 \text{ kWh/m}^2$ (the sun is not expected to shine much)
- Current battery state of charge $< 40\%$ (reserves are too low to cover the deficit)

If only one condition is true, a **warning** is raised. If neither, severity is 'none'. When the alert is active, a prioritized load-shedding list is returned to guide operators on what to switch off first.

Load Shedding Priority	Explanation
1. Defer HVAC and water heating	High-wattage comfort loads that can wait
2. Reduce secondary lighting	Non-critical illumination
3. Limit washing/cooking appliances	Can be delayed to off-peak hours
4. Critical loads only	Keep communications, refrigeration, security running

4.3 — `analytics_service.py`

Reads up to 5,000 stored prediction records from SQLite and computes:

- **Model Metrics:** count, mean, min, max, standard deviation of all stored GHI predictions
- **Seasonal Comparison:** for each record, determines if it belongs to the dry or rainy season based on the timestamp month and the city-specific season rules, then computes average GHI per season

4.4 — report_service.py

Builds FR-9 seasonal reports. Two outputs:

- **JSON report** (GET /api/reports/seasonal): summary of dry vs rainy averages
- **CSV export** (GET /api/reports/seasonal/csv): detailed row-by-row data with season label, month, prediction, temperature, humidity, wind, precipitation. Downloadable in the browser.

4.5 — weekly_report_service.py

Generates a **7-day (168-hour) PDF report** using the reportlab library. The PDF contains:

- Solar Energy Summary table: average/peak/min GHI, total panel energy, sample hours
- Microgrid Summary table: battery config, energy balances, self-sufficiency percentage
- Hourly Solar Forecast table (168 rows): time, GHI, temperature, humidity, wind, rain
- Hourly Microgrid Schedule table: time, solar kWh, load, battery SoC, import, export, status

The PDF is generated in-memory as bytes (BytesIO) and returned directly as a binary HTTP response, so it downloads immediately in the browser without saving to disk.

Chapter 5 — Machine Learning Pipeline

5.1 — Data Source (NASA POWER)

Training data comes from **NASA POWER** (Prediction of Worldwide Energy Resources), a public dataset providing daily/hourly meteorological and solar data globally. Variables downloaded for both Douala and Maroua (2000–2022):

- ALLSKY_SFC_SW_DWN — All-sky surface downward shortwave flux (W/m^2) — the TARGET variable
- T2M — Temperature at 2 metres ($^{\circ}\text{C}$)
- RH2M — Relative Humidity at 2 metres (%)
- WS10M — Wind speed at 10 metres (m/s)
- PRECTOTCORR — Bias-corrected precipitation (mm)

5.2 — Data Preprocessing (Data_Preprocessing.ipynb)

- Load raw NASA POWER CSV files (Douala_Dataset.csv, Maroua_Dataset.csv)
- Convert -999 sentinel values to NaN (NASA uses -999 for missing data)
- Drop rows with missing target variable (ALLSKY_SFC_SW_DWN)
- Forward-fill remaining missing values
- Parse datetime columns; add YEAR, MO, DY, HR columns
- Save cleaned datasets to Data/Processed/

5.3 — Exploratory Data Analysis (Exploratory_analysis.ipynb)

- Distribution plots: GHI values peak around midday, near zero at night
- Correlation heatmap: T2M moderately correlated with GHI, RH2M negatively correlated
- Seasonal patterns: Maroua has higher peak GHI; Douala more uniform due to clouds
- Time-of-day patterns: GHI rises from 6 AM, peaks 11 AM–1 PM, falls to 0 by 6 PM

5.4 — Feature Engineering (feature_model_II.ipynb)

Model II was specifically designed to avoid **solar leakage** — using features that are themselves derived from solar radiation (like sunshine duration or cloud cover) would make the model seem more accurate than it really is in production, because those features would not be available as forecast inputs.

Model II uses only: weather measurements (T2M, RH2M, WS10M, PRECTOTCORR), datetime fields (YEAR, MO, DY, HR), and engineered time/season features. **No solar-derived variables** are used as inputs.

5.5 — Model Training & Comparison

Three model types were trained for each city:

Model	Description	Library
Random Forest	Ensemble of 100–500 decision trees; each tree trained on a random subset of features	scikit-learn
XGBoost	Gradient Boosting with regularization. Builds trees sequentially; each corrects errors of the previous	xgboost
LSTM	Long Short-Term Memory recurrent neural network; designed for sequential time-series data	TensorFlow/Keras

Performance Results — Douala Model II:

Model	MAE (W/m ²)	RMSE (W/m ²)	R ²
Random Forest	24.81	54.72	0.9476
XGBoost	24.83	53.32	0.9503
LSTM	28.58	56.80	0.9434

Performance Results — Model II Best (Summary):

City	Best Model	MAE (W/m ²)	RMSE (W/m ²)	R ²
Douala	XGBoost	24.83	53.32	0.9503
Maroua	XGBoost	26.55	60.20	0.9657

Metrics explained: MAE (Mean Absolute Error) = average absolute difference between predicted and actual GHI. RMSE (Root Mean Square Error) = penalizes large errors more. R² = coefficient of determination; 1.0 is perfect; 0.95+ means the model explains 95% of the variance in solar irradiance — excellent for a weather-driven ML model.

5.6 — Why XGBoost Won

- Slightly better RMSE than Random Forest for both cities
- Much faster inference than LSTM (no recurrent computation)
- Works natively with tabular data (one row per hour); LSTM needs windowed sequences
- Built-in regularization (L1/L2) prevents overfitting
- joblib serialization of XGBoost models is compact and fast to load

5.7 — Model Validation (Model_validation.ipynb)

- Train-test split: 80% train, 20% test on chronological order (no shuffle — avoids data leakage)
- Cross-validation on the training set to tune hyperparameters

- Residual analysis: plot of predicted vs actual values, residual distribution
- Feature importance plot: which of the 21 features contribute most to predictions

Chapter 6 — Frontend Pages & Components

The frontend is a single-page application (SPA) built with **React 18** and bundled with **Vite**. Routing is handled by **react-router-dom v6**. Charts use **Recharts**. All API calls go through a single `client.js` module.

6.1 — main.jsx

Entry point. Renders the React app inside a **BrowserRouter** (from `react-router-dom`) which enables client-side URL routing. Attaches to the `#root` div in `index.html`.

6.2 — App.jsx

Defines all routes. The `Layout` component is a parent route that always renders (providing the sidebar). Child routes replace the placeholder inside `Layout` with the appropriate page component.

Route Path	Component Rendered
/ (index)	Dashboard — the default home page
/prediction	Prediction — manual forecast runner
/history	History — SQLite records table
/microgrid	Microgrid — optimization simulation
/reports	Reports — seasonal + weekly downloads
/about	About — static project description

6.3 — Layout.jsx (Sidebar Shell)

Renders the left sidebar with brand name 'SolarPredict', navigation links, and a footer. Uses `react-router-dom` `NavLink` which automatically adds the 'active' CSS class when the current URL matches the link's path. The component is where page content renders.

6.4 — Dashboard.jsx (FR-8)

The most complex page. Calls `GET /api/dashboard` on load and on every city change. Displays:

- **StatCards**: Temperature, Humidity, Wind Speed, Precipitation, Current GHI
- **RecommendationBadge**: Color-coded microgrid advice (green/orange/red)
- **24-Hour GHI LineChart** (Recharts) showing the forecast curve
- **BatteryGauge**: Visual state-of-charge bar with min/max markers
- **Seasonal Comparison BarChart**: Dry vs rainy season averages
- **Model Performance Metrics**: count, mean, min/max, std deviation
- **Microgrid Schedule Table**: 24 hourly rows with solar, load, SoC, status
- **AlertBanner**: FR-7 alert if active — shows severity, priorities

The dashboard also listens for **microgrid-settings-changed** browser events. When the user changes battery/panel settings on the Microgrid page, those changes are saved to `localStorage`, an event fires, and the Dashboard re-fetches with the new parameters.

6.5 — Prediction.jsx

User selects city and forecast hours (1–168), clicks Run Prediction. This posts to `POST /api/predict`. Results are shown as a `LineChart` and a table of up to 12 rows showing time, GHI, temperature, humidity, wind, rain, and microgrid status.

6.6 — History.jsx

Calls `GET /api/history` on load and when city filter changes. Displays all stored prediction records in a table sorted newest first. Shows temperature, humidity, wind, precipitation, prediction (W/m^2), and timestamp for each record.

6.7 — Microgrid.jsx

Form with city, battery capacity (kWh), daily load (kWh), and panel area (m^2). On submit, calls `POST /api/microgrid/optimize`. Shows a `BarChart` of solar generation vs load vs grid import/export per hour. Also displays a summary of self-sufficiency % and a schedule table.

Settings are saved to **localStorage** with 400ms debounce (`useEffect` + `setTimeout`). A custom browser event 'microgrid-settings-changed' is dispatched so `Dashboard.jsx` picks up the change immediately without a page reload.

6.8 — Reports.jsx (FR-9)

Two report sections:

- **Weekly PDF:** Calls `GET /api/reports/weekly/pdf`, receives binary blob, creates a temporary `<a>` element with the blob URL, and simulates a click to trigger download.
- **Seasonal Report:** Calls `GET /api/reports/seasonal` to get summary JSON. Shows dry vs rainy stat cards, a `BarChart`, and total record count.
- **CSV Download:** Calls `GET /api/reports/seasonal/csv`, same blob-download technique.
- **Chart PNG Export:** Uses `XMLSerializer` to convert the SVG chart to a string, draws it onto an HTML5 Canvas (adding a dark background), then downloads as PNG.

6.9 — Reusable Components

Component	Props	Purpose
StatCard	label, value, unit, icon, variant	Displays one metric in a card. 'accent' variant adds amber background.
RecommendationBadge	status, recommendation, level	Colored advice card. Level controls background: high=green, moderate=orange, low=red.
AlertBanner	alert (object)	Shows alert severity strip at top of dashboard. Hidden when no alert.

BatteryGauge	socPct, minPct, maxPct, currentHour, batteryCapacityKwh	Progress bar with color zones (critical/warning/normal/full) and marker lines at 20% min and 95% max.
--------------	--	---

6.10 — api/client.js

Single module that wraps all `fetch()` calls. All requests go through the **request()** helper which reads `VITE_API_URL` (for production deployment) or falls back to `/api` (proxied by Vite to `localhost:8000` during development). Error handling normalizes FastAPI's validation arrays and generic 500 errors into readable messages.

6.11 — microgridSettings.js

`localStorage` persistence layer for microgrid parameters. Ensures that settings entered on the Microgrid page are remembered across page reloads and shared with the Dashboard page. The custom event mechanism (`window.dispatchEvent`) allows cross-component communication without a global state manager like Redux.

Chapter 7 — API Endpoints Reference

The FastAPI backend exposes these endpoints. You can explore them interactively at <http://localhost:8000/docs> (Swagger UI) after starting the server.

Method	Endpoint	Description
GET	/	Root welcome message
GET	/api/health	Status check + city list
GET	/api/cities	All city metadata
GET	/api/dashboard	Full FR-8 dashboard payload
GET	/api/dashboard/basic	Lightweight dashboard (legacy)
POST	/api/predict	Hourly GHI forecast
GET	/api/history	Stored predictions
GET	/api/microgrid/recommendation	Rule-based advice for one value
POST	/api/microgrid/optimize	Full battery/grid simulation
GET	/api/reports/seasonal	Seasonal comparison JSON (FR-9)
GET	/api/reports/seasonal/csv	Download CSV (FR-9)
GET	/api/reports/weekly/pdf	Download 7-day PDF

7.1 — Dashboard Endpoint Deep Dive (GET /api/dashboard)

This is the most important and complex endpoint. Here is exactly what it does step by step:

1. Validate city and microgrid parameters (raise 400 if invalid)
2. Call `fetch_hourly_weather(city, 24)` → get 24 weather records from Open-Meteo
3. Call `build_feature_dataframe()` → build a 24-row × 21-column DataFrame
4. Call `predict(city, feature_df)` → get 24 predicted GHI values from XGBoost
5. Get current hour weather snapshot from `weather_records[0]`
6. Call `get_microgrid_recommendation()` on the current prediction
7. Call `get_dashboard_stats()` → aggregate SQLite stats
8. Check if latest saved prediction matches current hour; if not, save to SQLite
9. Call `run_microgrid_optimization()` → simulate 24-hour battery/grid schedule
10. Call `evaluate_alert()` → check FR-7 thresholds
11. Call `get_dashboard_analytics()` → compute metrics and seasonal comparison
12. Build and return `EnhancedDashboardResponse` with all sub-objects

7.2 — Microgrid Optimize Request/Response Example

Request body (POST /api/microgrid/optimize):

```
{ "city": "douala", "battery_capacity_kwh": 20, "daily_load_kwh": 35,  
  "panel_area_m2": 12, "panel_efficiency": 0.18, "hours": 24 }
```

Response (abbreviated):

```
{ "city": "douala", "battery_capacity_kwh": 20, "predicted_solar_kwh": 12.4,  
  "grid_import_kwh": 5.2, "grid_export_kwh": 0.8, "self_sufficiency_pct": 85.1,  
  "current_battery_soc_pct": 52.3, "schedule": [  
    { "hour": 0, "datetime": "2026-06-23T06:00:00", "prediction": 450.2,  
      "solar_generation_kwh": 0.972, "load_kwh": 1.458, "battery_soc_pct": 48.5,  
      "charge_kwh": 0, "discharge_kwh": 0.486, "grid_import_kwh": 0, ... },  
    ... 23 more hours  
  ] }
```

Chapter 8 — Database Schema & Queries

8.1 — predictions Table Schema

Column	SQL Type	Description
id	INTEGER PRIMARY KEY AUTOINCREMENT	Unique record identifier
city	TEXT NOT NULL	'douala' or 'maroua' — always stored lowercase
temperature	REAL NOT NULL	Air temperature at 2m (°C) from Open-Meteo
humidity	REAL NOT NULL	Relative humidity at 2m (%)
wind_speed	REAL NOT NULL	Wind speed at 10m (m/s)
precipitation	REAL NOT NULL	Precipitation (mm)
prediction	REAL NOT NULL	Predicted solar irradiance (W/m ²) — clamped to ≥ 0
timestamp	TEXT NOT NULL	ISO 8601 datetime string of the forecast hour

8.2 — Key SQL Queries

Get latest prediction for a city:

```
SELECT * FROM predictions WHERE city='douala' ORDER BY timestamp DESC LIMIT 1
```

Get prediction history with city filter:

```
SELECT id, city, temperature, humidity, wind_speed, precipitation, prediction,
timestamp
FROM predictions WHERE city=? ORDER BY timestamp DESC LIMIT 100
```

Dashboard stats — average per city:

```
SELECT city, AVG(prediction) AS avg_prediction FROM predictions GROUP BY city
```

Check if table exists (migration logic):

```
SELECT name FROM sqlite_master WHERE type='table' AND name='predictions'
```

8.3 — Why SQLite?

- Zero-configuration — no separate database server needed
- File-based (solar_predict.db) — easy to inspect, backup, and version
- Python standard library includes sqlite3 — no extra dependency
- Sufficient for a single-server, moderate-load academic project
- For production scale, you could swap to PostgreSQL by replacing sqlite3 with psycopg2

8.4 — inspect_db.py

A utility script in the backend/ folder that connects to solar_predict.db, lists all tables, and shows the first few rows. Useful for debugging during development.

Chapter 9 — Microgrid Optimization Algorithm

The microgrid simulation in **microgrid.py** implements a greedy hourly dispatch strategy. It is not a linear program — it makes locally optimal decisions hour by hour without look-ahead. This is intentional for simplicity and explainability.

9.1 — Parameters

Parameter	Explanation
battery_capacity_kwh	Maximum energy the battery can store (e.g., 20 kWh)
daily_load_kwh	Total energy consumed per day. Divided by 24 to get hourly_load (e.g., 35/24 $\approx$ 1.46 kWh/h)
panel_area_m2	Total solar panel area in square metres (e.g., 12 m ²)
panel_efficiency	Fraction of sunlight converted to electricity (0.18 = 18%)
BATTERY_SOC_MIN_PCT (20%)	Battery never discharges below 20% — protects battery life
BATTERY_SOC_MAX_PCT (95%)	Battery never charges above 95% — prevents overcharge
BATTERY_SOC_DEFAULT_PCT (50%)	Simulation starts at 50% state of charge

9.2 — Irradiance to kWh Conversion

The model predicts irradiance in **W/m²**. To get kWh of energy generated per hour:

```
solar_kwh = max(0.0, irradiance (W/m2) × panel_area (m2) × efficiency / 1000)
```

Example: 500 W/m² × 12 m² × 0.18 / 1000 = 1.08 kWh in one hour

9.3 — Hourly Dispatch Logic

For each of the 24 hours, the algorithm:

- Compute solar_kwh for the hour using the formula above
- Compute net = solar_kwh – hourly_load
- If net $\geq$ 0 (solar surplus):
 - → Charge the battery: charge_room = max_soc – current_soc
 - → charge_kwh = min(net, charge_room) — don't overfill
 - → surplus = net – charge_kwh
 - → If surplus > 0: export to grid (grid_export += surplus)
- If net < 0 (solar deficit):
 - → Discharge the battery: discharge_room = current_soc – min_soc
 - → discharge_kwh = min(|net|, discharge_room) — don't deplete past min

- → remaining = |net| – discharge_kwh
- → If remaining > 0: import from grid (grid_import += remaining)
- Record all values in a MicrogridHour object

9.4 — Self-Sufficiency Calculation

```
self_sufficiency_pct = min(100, (total_load - grid_import) / total_load × 100)
```

100% = all energy came from solar/battery (no grid import needed)

50% = half the load was covered by solar+battery, half by grid

9.5 — run_microgrid_optimization() vs simulate_microgrid()

simulate_microgrid() is a pure synchronous function — it only takes already-computed weather records and prediction values. This makes it reusable from the dashboard (which already has predictions) without re-fetching weather.

run_microgrid_optimization() is async and orchestrates: if weather/predictions are not provided, it fetches weather and runs XGBoost, then calls **simulate_microgrid()**. The dashboard passes pre-computed data to avoid double-fetching.

Chapter 10 — Alert System (FR-7)

The alert system is implemented in `alert_service.py` and integrated into the dashboard endpoint.

10.1 — Alert Conditions

Condition	Value / Threshold
ALERT_FORECAST_HOURS	6 hours — look 6 steps ahead in the forecast
ALERT_SOLAR_THRESHOLD_KWH_M2	0.5 kWh/m ² — 6-hour cumulative GHI per m ²
ALERT_BATTERY_SOC_PCT	40% — battery must also be below 40% for critical alert

10.2 — GHI to kWh/m² Conversion

```
For each of the 6 forecast hours:
energy_density (kWh/m2) = max(0, irradiance_wm2 / 1000)

Sum across 6 hours → solar_6h_kwh_m2
If this is < 0.5 AND battery < 40% → CRITICAL ALERT (active=True)
If only one condition true → WARNING
Neither condition → OK (severity='none')
```

10.3 — AlertBanner Component

On the Dashboard page, the AlertBanner component renders differently based on severity:

- **none:** Banner is not shown at all (return null)
- **warning:** Orange-themed banner with watch message
- **critical:** Red-themed banner with 'Active Alert' heading and load-shedding list

Chapter 11 — Report Generation (FR-9)

11.1 — Seasonal Report (JSON + CSV)

The seasonal report reads all stored predictions (up to 10,000 rows) from SQLite. For each record, it determines whether the timestamp falls in the dry or rainy season using the same `_season_flags()` function from `features.py`:

- Douala dry season: months 12, 1, 2 (December–February)
- Douala rainy season: months 3–11
- Maroua dry season: months 1–5, 10–12 (outside rainy season)
- Maroua rainy season: months 6, 7, 8, 9 (June–September)

The CSV export uses Python's `csv.DictWriter` to write all seasonal rows as a properly formatted CSV that can be opened in Excel or any spreadsheet tool.

11.2 — Weekly PDF Report

Uses the **reportlab** library to build a professional PDF with:

- SimpleDocTemplate on A4 paper with custom margins
- Paragraph styles for title, subtitle, section headings, body
- Table objects with alternating row backgrounds (`TableStyle`)
- PageBreak objects to structure content across pages
- BytesIO buffer — the PDF bytes are written to memory, not disk

The PDF is returned as a FastAPI Response with content-type 'application/pdf' and Content-Disposition 'attachment; filename=...' so the browser downloads it immediately.

Chapter 12 — How to Run the Project

12.1 — Prerequisites

- Python 3.11 or higher (check with: `python --version`)
- Node.js 18+ (check with: `node --version`)
- npm (check with: `npm --version`)
- Git (optional, for version control)

12.2 — Start the Backend

```
# From the SolarPredict/ root directory:
cd backend
pip install -r requirements.txt
python -m uvicorn app.main:app --reload --port 8000

# Expected output:
INFO: Uvicorn running on http://127.0.0.1:8000
INFO: Started reloader process
INFO: Application startup complete.
```

Then open: <http://127.0.0.1:8000/docs> to see the interactive Swagger UI.

12.3 — Start the Frontend

```
# In a new terminal, from SolarPredict/:
cd frontend
npm install # first time only – installs React, Recharts, etc.
npm run dev

# Expected output:
VITE v5.x ready in 300ms
Local: http://localhost:5173/
```

Then open: <http://localhost:5173> to see the dashboard.

12.4 — Typical Usage Flow

Page	What To Do
Dashboard (home page)	Select Douala or Maroua → loads live GHI forecast, battery SoC, alert, seasonal comparison automatically
Prediction page	Select city, set hours (1–168), click Run Prediction → see forecast chart and table
Microgrid page	Set battery/load/panel parameters → click Optimize → see energy balance chart
History page	Filter by city → see all stored prediction records from SQLite

Reports page	Click Download Weekly PDF → get 7-day PDF; Download CSV → get seasonal data
About page	Static explanation of the project, tech stack, and workflow

Chapter 13 — Tests to Run & Demonstrate

These are the key tests you should be prepared to run or explain during your defense. They cover both functional correctness and edge cases.

13.1 — API Health Tests (run in browser or curl)

```
GET http://localhost:8000/api/health
Expected: { 'status': 'ok', 'cities': ['douala', 'maroua'] }

GET http://localhost:8000/api/cities
Expected: List of 2 objects with id, name, lat, lon
```

13.2 — Prediction API Test

```
POST http://localhost:8000/api/predict
Body: { "city": "douala", "hours": 6 }

Expected: city='douala', model='XGBoost Model II', 6 predictions,
each with datetime, prediction (≥0), weather snapshot, microgrid advice

# Test invalid city:
Body: { "city": "yaoundé", "hours": 24 }
Expected: HTTP 400 – 'Unsupported city: yaoundé'
```

13.3 — Dashboard Test

```
GET http://localhost:8000/api/dashboard?city=maroua
Expected: forecast_24h (24 items), microgrid_schedule (24 items),
alert object, model_metrics, seasonal_comparison
```

13.4 — Microgrid Optimization Tests

```
# Test normal case:
POST /api/microgrid/optimize
{ "city": "douala", "battery_capacity_kwh": 20, "daily_load_kwh": 35,
  "panel_area_m2": 12, "panel_efficiency": 0.18, "hours": 24 }
Expected: 24-item schedule, self_sufficiency_pct > 0

# Test SoC limits are respected:
Verify in schedule: battery_soc_pct never < 20 and never > 95

# Test invalid panel efficiency:
{ ..., "panel_efficiency": 1.5 }
Expected: HTTP 422 Unprocessable Entity (Pydantic validation: le=1)
```

13.5 — Alert System Tests

```
# Test alert threshold at recommendation endpoint:
GET /api/microgrid/recommendation?prediction=300
Expected: { status: 'Low Solar Energy', level: 'low' }

GET /api/microgrid/recommendation?prediction=500
Expected: { status: 'Moderate Solar Energy', level: 'moderate' }
```

```
GET /api/microgrid/recommendation?prediction=800
Expected: { status: 'High Solar Energy', level: 'high' }
```

13.6 — History & Database Tests

```
# After running predictions:
GET /api/history?city=douala&limit;=10
Expected: ≤10 records with all columns present

GET /api/history?city=invalidcity
Expected: HTTP 400 – 'Unsupported city'
```

13.7 — Feature Engineering Unit Test (Python)

```
from datetime import datetime
from backend.app.features import build_model_ii_row

dt = datetime(2026, 6, 23, 12, 0) # noon in June
row = build_model_ii_row(dt, t2m=28.0, rh2m=75.0, ws10m=3.0,
prectotcorr=0.0, city='douala')

assert row['HR'] == 12
assert row['is_daytime'] == 1 # 6 <= 12 <= 18
assert row['is_rainy'] == 0 # no rain
assert row['season_dry'] == 0 # June is rainy in Douala
assert row['season_rainy'] == 1
assert len(row) == 21 # exactly 21 features
```

13.8 — Report Tests

```
GET /api/reports/seasonal?city=douala
Expected: summary with dry_season and rainy_season buckets, record_count

GET /api/reports/seasonal/csv?city=douala
Expected: Content-Type: text/csv, file downloads

GET /api/reports/weekly/pdf?city=douala
Expected: Content-Type: application/pdf, file downloads
```

13.9 — CORS Test

The frontend at localhost:5173 communicates with the backend at localhost:8000. CORS is configured to allow this. If CORS were missing, the browser would block requests. You can demonstrate: open browser devtools (F12) → Network tab → run any prediction → check that the request succeeds with no CORS error.

13.10 — Cache Test

Weather data is cached for 5 minutes. You can demonstrate by:

- Clicking Refresh on Dashboard (first time: real API call ~500ms)
- Clicking Refresh again within 5 minutes — response is instant (from cache)

- Waiting 5+ minutes and refreshing again — new API call (slower)

Chapter 14 — Defense Questions & Model Answers

Q: GENERAL: What problem does SolarPredict solve?

A: Solar energy is intermittent and unpredictable. Without forecasting, microgrid operators cannot plan when to charge batteries, when to use the grid, or when to shed load. SolarPredict provides hourly GHI forecasts and translates them into actionable microgrid decisions, reducing grid dependency and preventing outages in Douala and Maroua.

Q: Why focus on Douala and Maroua specifically?

A: These two cities represent contrasting climate profiles. Douala is coastal and equatorial with moderate but consistent solar resource and high humidity. Maroua is in the Sahelian Far North with high solar potential and a clear dry/rainy season split. Together they demonstrate that the model generalizes across Cameroonian climate zones.

Q: What is GHI (Global Horizontal Irradiance)?

A: GHI measures the total solar radiation received per unit area on a horizontal surface, in Watts per square metre (W/m^2). It includes both direct sunlight and diffuse (scattered) light. It is the standard metric for solar energy resource assessment. Our target variable is ALLSKY_SFC_SW_DWN from NASA POWER, which is the all-sky (includes cloud effects) surface downward shortwave irradiance — equivalent to GHI.

Q: Why did you choose XGBoost over Random Forest or LSTM?

A: XGBoost achieved the best RMSE on both cities (53.32 W/m^2 for Douala, $R^2=0.95$; 60.20 W/m^2 for Maroua, $R^2=0.97$). Random Forest was close but slightly worse. LSTM was noticeably worse because the model treats each hour independently (no temporal window), removing LSTM's advantage. XGBoost is also faster to train, easier to serialize with joblib, and has built-in regularization to prevent overfitting.

Q: What does $R^2=0.95$ mean in practice?

A: R^2 (R-squared) is the coefficient of determination. A value of 0.95 means the model explains 95% of the variance in solar irradiance. The remaining 5% is unexplained — likely due to sudden cloud cover events or measurement noise. For a weather-driven regression task, 0.95+ is considered excellent performance.

Q: What is 'solar leakage' and how did you avoid it?

A: Solar leakage means using features that are themselves derived from solar radiation as model inputs. For example, cloud cover fraction or sunshine duration are correlated with GHI but are not available as reliable forecasts. Model I had this issue. Model II uses only raw weather variables (temperature, humidity, wind, rain) and datetime features — inputs that are available from standard weather forecast APIs. This makes the model production-ready without inflating performance metrics.

Q: Why use cyclic encoding (sin/cos) for time features?

A: Categorical time features like hour (0–23) have a circular structure: hour 23 and hour 0 are adjacent, but linear encoding gives them a distance of 23. Sin/cos encoding maps hours onto a unit circle, so 23 and 0 are geometrically close. This helps tree models learn daily and seasonal patterns without creating artificial boundaries at midnight or December/January.

Q: Why use log1p transforms on precipitation and wind speed?

A: Both variables are highly right-skewed: most values are near zero (e.g., 0–2 mm rain) but occasionally spike to very large values (50+ mm). $\log(1 + x)$ compresses the tail, making the distribution more symmetric and helping XGBoost learn more efficient split points. log1p is used instead of log to handle zero values ($\log(0) = -\infty$, but $\log1p(0) = 0$).

Q: What is the train/test split and why?

A: The data is split 80% training, 20% testing in chronological order (not random). Chronological splitting is critical for time-series: if you shuffle, future data leaks into training (the model sees tomorrow's weather while predicting yesterday's). Chronological splitting ensures the test set represents genuinely unseen future periods.

Q: Why FastAPI and not Flask or Django?

A: FastAPI is purpose-built for modern async APIs. It provides automatic request validation via Pydantic (no manual if/else validation), auto-generates Swagger/OpenAPI docs at /docs, uses Python's async/await natively for non-blocking HTTP calls to Open-Meteo, and has excellent performance (based on Starlette+ASGI). Flask lacks native async support; Django is heavier and more suited for full-stack web apps with templates.

Q: What is Pydantic and why is it important?

A: Pydantic is a data validation library. In FastAPI, every request body and response model is a Pydantic BaseModel. When a request arrives, Pydantic automatically checks that all required fields are present and the right type. If not, it returns a clear 422 error. When returning responses, it serializes Python objects to JSON. This removes all manual validation/serialization code from routes.

Q: How does the weather caching work?

A: The `_weather_cache` dict in `weather.py` maps (city, forecast_hours) tuples to (fetch_time, records) pairs. Before making an HTTP request, the code checks if a cached result exists and if it is less than 300 seconds (5 minutes) old. If yes, the cached data is returned immediately. This prevents hammering Open-Meteo on every dashboard refresh. The cache lives in process memory, so it resets when the server restarts.

Q: What happens if Open-Meteo is down?

A: The `fetch_hourly_weather()` function raises an `httpx` exception. In `main.py`, all calls to `weather` are wrapped in `try/except` blocks that catch `Exception` and raise `HTTPException(status_code=502)`. The frontend `client.js` catches HTTP errors and displays a user-friendly message like 'Backend unavailable' instead of a blank screen.

Q: Why does main.py preload models at startup?

A: Loading a .pkl file from disk and deserializing it takes 100–500ms for large models. If models were loaded on first request, the first user to predict would experience a noticeable delay.

Preloading at startup (the `on_startup` event) ensures models are in memory (in the `_models` dict) before any request arrives. Subsequent requests are instant (cache hit).

Q: What is CORS and why do you configure it?

A: CORS (Cross-Origin Resource Sharing) is a browser security mechanism. By default, browsers block JavaScript from making requests to a different origin (host:port). The frontend at `localhost:5173` calling the backend at `localhost:8000` is a different origin, so without CORS middleware, all API calls would fail. FastAPI's `CORSMiddleware` adds the `Access-Control-Allow-Origin` header to responses, telling the browser to permit the cross-origin requests.

Q: What is the database migration strategy?

A: When the app starts, `init_db()` checks if the predictions table exists and compares its columns to the expected schema. If columns differ (e.g., an old version used `'t2m'` instead of `'temperature'`), the table is renamed to `predictions_legacy`, a new table is created with the correct schema, data is copied with column renaming, and the legacy table is dropped. This allows schema evolution without losing data.

Q: Explain the microgrid optimization algorithm.

A: It is a greedy hourly dispatch algorithm. For each hour: (1) compute solar energy from irradiance using $\text{solar_kwh} = \text{GHI} \times \text{area} \times \text{efficiency} / 1000$. (2) Compare to hourly load. (3) If solar exceeds load, charge the battery up to 95% SoC, export any remaining surplus to the grid. (4) If solar falls short, discharge battery down to 20% SoC, import remaining deficit from grid. $\text{Self-sufficiency} = (\text{total_load} - \text{grid_import}) / \text{total_load} \times 100$.

Q: Why are the SoC limits 20% and 95%?

A: These are standard battery management thresholds. Discharging a lead-acid or lithium battery below 20% (deep discharge) significantly shortens its lifespan. Charging above 95% can cause overheating and chemical degradation. The 5% headroom at the top also allows for charge absorption during the CV (constant voltage) phase. These values are stored as constants in `config.py` to make them easy to adjust.

Q: What does self-sufficiency percentage mean?

A: Self-sufficiency = the fraction of total load met without importing from the grid. 100% means the system is entirely off-grid during the simulation period. 70% means 70% of energy came from solar+battery, 30% from the grid. It is a key performance indicator for microgrid design.

Q: Why React and not Vue or plain HTML?

A: React's component model (Layout, StatCard, BatteryGauge, etc.) allows reusable, maintainable UI pieces. Vite provides extremely fast development server with Hot Module Replacement (HMR). Recharts integrates natively with React's rendering cycle for interactive charts. The routing (`react-router-dom`) enables a proper SPA with URL navigation without full page reloads.

Q: How does the Dashboard stay in sync with Microgrid settings?

A: When the user updates battery/panel settings on the Microgrid page, `saveMicrogridSettings()` writes to `localStorage` and dispatches a custom browser event `'microgrid-settings-changed'`.

Dashboard.jsx has a useEffect that listens for this event and also for the 'storage' event (for cross-tab sync). When either fires, it re-reads localStorage and re-fetches the dashboard with the new parameters.

Q: How does the PDF/CSV download work on the frontend?

A: The downloadWeeklyPdf() and downloadSeasonalCsv() functions in client.js call fetch() on the binary endpoint. The response is read as a Blob (binary large object). A temporary <a> element is created with the blob URL as href and the desired filename as download attribute. Simulating a click on this element triggers the browser's download dialog. The blob URL is then revoked to free memory.

Q: What does Recharts provide and how is it used?

A: Recharts is a chart library built on React and D3. It provides LineChart, BarChart, ResponsiveContainer, XAxis, YAxis, Tooltip, Legend, Line, Bar components. Each chart wraps data in JSX, making it declarative. ResponsiveContainer makes charts resize automatically. The GHI line chart on Dashboard/Prediction uses LineChart; the seasonal/energy balance charts use BarChart.

Q: How would you deploy this to production?

A: The project includes a vercel.json configuration. The backend (FastAPI) would deploy as a serverless function or a Python web service on Vercel. The frontend (React/Vite) builds to static files (npm run build) and deploys as a static site. Environment variables VERCEL_URL and FRONTEND_URL are read by main.py to add the production domain to CORS origins. Alternatively, both could be containerized with Docker and deployed to any cloud provider (AWS, GCP, Azure).

Q: Why is SQLite used in production instead of PostgreSQL?

A: SQLite is appropriate for a final year project with moderate data volumes and single-server deployment. It requires no separate database server, reducing infrastructure complexity. For a production microgrid monitoring system with multiple concurrent users, PostgreSQL would be recommended for concurrent write safety, better indexing, and connection pooling.

Q: Walk me through a complete prediction request from browser to response.

A: 1. User clicks 'Run Prediction' on Prediction.jsx. 2. api.predict() sends POST /api/predict with {city, hours}. 3. FastAPI validates PredictionRequest via Pydantic. 4. predict_solar() route calls fetch_hourly_weather(city, hours) → Open-Meteo API returns hourly arrays. 5. For each hour, build_model_ii_row() computes 21 features from weather + datetime. 6. build_feature_dataframe() creates a pandas DataFrame with correct column order. 7. predict() calls model.predict(df) → XGBoost returns raw scores → clamped to ≥ 0 . 8. For each prediction, get_microgrid_recommendation() assigns status/level. 9. save_predictions() inserts all rows to SQLite. 10. PredictionResponse is returned as JSON. 11. Frontend renders LineChart and table.

Q: What would you improve if you had more time?

A: 1. Add real-time WebSocket updates on the dashboard instead of manual refresh. 2. Implement user authentication so multiple users can have separate prediction histories. 3. Use PostgreSQL for production-grade data storage. 4. Add a more sophisticated optimization

algorithm (linear programming with PuLP/scipy) for microgrid dispatch with look-ahead. 5. Add more Cameroonian cities. 6. Implement model retraining pipeline with new data. 7. Add unit tests with pytest covering all service functions.

Q: What are the limitations of your model?

A: 1. Training data is historical (2000–2022); the model cannot predict unprecedented weather events (extreme storms, etc.). 2. Open-Meteo weather forecasts themselves have uncertainty — the model's accuracy is bounded by how accurate the weather forecast is. 3. The microgrid model assumes a simple single-battery setup; real microgrids often have multiple storage units and variable loads. 4. The model was trained on daily granularity NASA data upsampled to hourly — ideally sub-hourly data would improve accuracy. 5. No real-time feedback loop — the model does not learn from operational data.

Appendix — Technology Stack Summary

Technology	Version	Role
Python	3.11+	Backend runtime language
FastAPI	0.115.6	REST API framework
Uvicorn	0.32.1	ASGI server (runs FastAPI)
Pydantic	2.10.3	Request/response validation
XGBoost	2.1.3	Machine learning inference
joblib	1.4.2	Model serialization (.pkl files)
pandas	2.2.3	Feature DataFrame construction
numpy	2.1.3	Numerical operations (log1p, etc.)
httpx	0.28.1	Async HTTP client for Open-Meteo
reportlab	4.2.5	PDF report generation
SQLite3	stdlib	Database (via Python standard library)
React	18.x	Frontend UI framework
Vite	5.x	Frontend build tool and dev server
react-router-dom	6.x	Client-side routing
Recharts	2.x	Chart library (LineChart, BarChart)
Open-Meteo API	Free tier	Real-time weather forecasts
NASA POWER	Historical	Training data source (2000–2022)

SolarPredict Final Year Project Defense Guide — Generated for project defense preparation. This document covers all aspects of the system: architecture, code, ML, API, frontend, tests, and expected defense questions.